

Session 4 — MDP Formulation

1 · What is an MDP?

Markov Decision Process (MDP)

An MDP is a mathematically precise formulation of the RL problem. It consists of:

$$\langle \mathcal{S}, \mathcal{A}, \mathcal{R}, p, \gamma \rangle$$

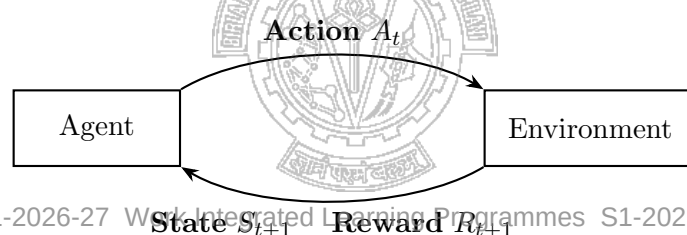
- $\mathcal{S}$ — finite set of **states**
- $\mathcal{A}$ — finite set of **actions**
- $\mathcal{R}$ — set of **rewards** (real-valued scalars)
- $p(s', r | s, a)$ — **dynamics function** (transition + reward probabilities)
- γ — **discount factor**, $0 \leq \gamma \leq 1$

The Markov Property: The future depends *only* on the current state, not on the history.

$$P(S_{t+1}, R_{t+1} | S_t, A_t) = P(S_{t+1}, R_{t+1} | S_0, A_0, \dots, S_t, A_t)$$

2 · The 3 Signals Between Agent and Environment

Three Signals — Must Know



Signal	Direction	Meaning
S_t (State)	Environment → Agent	Current situation the agent observes
A_t (Action)	Agent → Environment	Decision taken at step t
R_{t+1} (Reward)	Environment → Agent	Scalar feedback after taking A_t in S_t

Dynamics function: $p(s', r | s, a) = \Pr\{S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a\}$

3 · Returns, Discounting, and G_t

Goal: Maximise the **expected return** G_t — not just the next reward, but the cumulative future reward.

Discounted Return

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Role of γ (discount factor):

γ	Effect	Use case
$\gamma = 0$	Only immediate reward matters (myopic)	Short-horizon tasks; immediate payoff
$0 < \gamma < 1$	Future rewards discounted exponentially	Most practical RL tasks
$\gamma = 1$	All future rewards count equally	Episodic tasks with finite horizon only

Why discount?

1. Mathematical convergence: guarantees G_t is finite when rewards are bounded.
2. Economic interpretation: a reward now is worth more than the same reward later (time preference).
3. Uncertainty: future transitions may not occur as expected; discounting hedges against this.

4 · Episodic vs. Continuing Tasks

Two Task Types		
	Episodic	Continuing
Termination	Has a terminal state T	Runs forever
Return formula	$G_t = \sum_{k=0}^{T-t-1} R_{t+k+1}$ (no γ needed)	$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ (need $\gamma < 1$)
Examples	Chess, Tic-Tac-Toe, maze	Trading bot, robot control
$\gamma = 1$ allowed?	Yes (finite sum)	No (infinite sum diverges)

Unified formula (using absorbing terminal state with zero reward):

$$G_t = \sum_{k=t+1}^T \gamma^{k-t-1} R_k \quad (\text{works for both; set } T = \infty \text{ for continuing})$$

5 · Recursive Formulation — G_t in terms of G_{t+1}

Key Recursion — used in every RL algorithm
$G_t = R_{t+1} + \gamma G_{t+1}$

Derivation (one step):

$$\begin{aligned}
 G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \\
 &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots) \\
 &= R_{t+1} + \gamma G_{t+1}
 \end{aligned}$$

Why is this useful?

- Allows computing G_t *backwards* from the terminal state — no need to store the full trajectory.
- Directly motivates the TD update: estimate $G_t \approx R_{t+1} + \gamma V(S_{t+1})$.
- The Bellman equations (next section) are built entirely on this recursion.

6 · Value Functions and Bellman Equation

State-value and action-value functions

innovate

achieve

lead

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s] = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma v_{\pi}(s')]$$

$$q_{\pi}(s,a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a] = \sum_{s',r} p(s',r|s,a) [r + \gamma v_{\pi}(s')]$$

The first equation is the **Bellman equation** for v_{π} .

Session 5 — Dynamic Programming (DP)

7 · Characteristics of DP

What is DP in the RL context?

DP refers to a collection of algorithms that use the **Bellman equations** to compute optimal value functions, given a **perfect model** of the environment.

Characteristics:

1. Requires a **complete model**: $p(s',r | s,a)$ must be fully known.
2. Computationally **expensive** for large state spaces, but **exact**.
3. Uses **bootstrapping**: updates are based on estimates of other states (not sampled rewards alone).
4. Finds the **optimal policy** π^* and optimal value function v^* .

When DP is applicable:

- State and action spaces are **finite** and **small**.
- The full transition model $p(s',r|s,a)$ is **known** (not needed to interact with real world).
- Examples: grid worlds, small board games, Jack's Car Rental, Gambler's Problem.
- Not applicable: large/continuous spaces (use function approximation), unknown model (use MC or TD).

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

8 · Key Terms (Policy Evaluation → Value Iteration)

Five Key Terms — Know Each Precisely

(1) Policy Evaluation (Prediction Problem)

Given policy π , compute $v_\pi(s)$ for all states. Use iterative sweeps:

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) [r + \gamma v_k(s')]$$

Repeat until $\max_s |v_{k+1}(s) - v_k(s)| < \theta$ (some small threshold).

(2) Policy Improvement (Control — one step)

Given v_π , form a *greedy* policy π' :

$$\pi'(s) = \arg \max_a \sum_{s',r} p(s', r|s, a) [r + \gamma v_\pi(s')]$$

Policy Improvement Theorem: $v_{\pi'}(s) \geq v_\pi(s)$ for all s (the new greedy policy is at least as good as the old one).

(3) Policy Iteration

Alternate: full Policy Evaluation → full Policy Improvement → repeat.

Converges to π^* in a finite number of iterations (finite MDP).

(4) Value Iteration

Combine one sweep of (partial) evaluation + improvement:

$$v_{k+1}(s) = \max_a \sum_{s',r} p(s', r|s, a) [r + \gamma v_k(s')]$$

No need to wait for full policy evaluation to converge between improvements. Faster in practice.

(5) Generalized Policy Iteration (GPI)

The general idea of *any* interleaving of evaluation and improvement, regardless of granularity. All RL algorithms are a form of GPI.

Asynchronous DP

Standard DP sweeps all states uniformly (synchronous). **Asynchronous DP** updates states in any order — it can focus on states that are most relevant (e.g., those the agent visits often).

- Useful for large state spaces.
- Still requires a full model.
- Converges as long as every state is updated infinitely often in the limit.

9 · Policy Iteration vs. Value Iteration

	Policy Iteration	Value Iteration
Evaluation step	Run full Bellman expectation until convergence	One Bellman <i>optimality</i> sweep
Improvement step	Full greedy improvement after evaluation	Combined into same sweep
Iterations	Fewer policy updates, each costly	More iterations but each cheap
Update per step	$v_{k+1}(s) = \sum_a \pi(a s) [\dots]$	$v_{k+1}(s) = \max_a [\dots]$
Convergence	Finite (finite MDP)	Geometric (γ^k contraction)
When to prefer	When evaluation is cheap; small MDP	When model is known but large

10 · Worked Example — Value Iteration (Race Car / 1-D Track)

Race Car Problem (Simplified)

innovate

achieve

lead

Setup: A car is on a 1-D track: states $s \in \{0, 1, 2, 3, 4\}$ (positions). State 4 = **Finish** (terminal).

Actions: **Fast** (move 2 positions forward, reward -1) or **Slow** (move 1 position forward, reward -1).

Goal: reach state 4 in minimum steps. Discount $\gamma = 0.9$. All transitions are deterministic.

Initialise: $v_0(s) = 0$ for all s . $v(4) = 0$ always (terminal).

Iteration 1 — Value Iteration sweep (apply $v_{k+1}(s) = \max_a [r + \gamma v_k(s')]$):

From state 3: Fast $\rightarrow 5$ (out of range/clamped to 4), Slow $\rightarrow 4$. Both reach finish: $r = -1$, $v_0(4) = 0$.

$$v_1(3) = \max(-1 + 0.9 \times 0, -1 + 0.9 \times 0) = -1$$

From state 2: Fast $\rightarrow 4$ ($r = -1$, $v_0(4) = 0$); Slow $\rightarrow 3$ ($r = -1$, $v_0(3) = 0$).

$$v_1(2) = \max(-1 + 0, -1 + 0) = -1$$

From state 1: Fast $\rightarrow 3$ ($r = -1$, $v_0(3) = 0$); Slow $\rightarrow 2$ ($r = -1$, $v_0(2) = 0$).

$$v_1(1) = -1$$

From state 0: Fast $\rightarrow 2$; Slow $\rightarrow 1$. Both: -1 .

$$v_1(0) = -1$$

Iteration 2:

From state 3: same as before $= -1$.

From state 2: Fast $\rightarrow 4$ ($-1 + 0 = -1$); Slow $\rightarrow 3$ ($-1 + 0.9 \times (-1) = -1.9$).

$$v_2(2) = \max(-1, -1.9) = -1 \Rightarrow \text{prefer Fast}$$

From state 1: Fast $\rightarrow 3$ ($-1 + 0.9 \times (-1) = -1.9$); Slow $\rightarrow 2$ ($-1 + 0.9 \times (-1) = -1.9$).

$$v_2(1) = -1.9$$

From state 0: Fast $\rightarrow 2$ ($-1 + 0.9 \times (-1) = -1.9$); Slow $\rightarrow 1$ ($-1 + 0.9 \times (-1.9) = -2.71$).

$$v_2(0) = \max(-1.9, -2.71) = -1.9 \Rightarrow \text{prefer Fast}$$

Emerging policy: always take **Fast** — reach finish in fewest steps. Values converge downward as they properly account for the number of steps to terminal.

11 · Generalized Policy Iteration (GPI) — The Big Picture

GPI — All RL is GPI

$$\pi \xrightarrow{\text{Evaluate}} v_\pi \xrightarrow{\text{Improve}} \pi' \xrightarrow{\text{Evaluate}} v_{\pi'} \xrightarrow{\text{Improve}} \dots \rightarrow \pi^*, v^*$$

- **Evaluation** drives v toward v_π (makes value consistent with current policy).
- **Improvement** drives π toward greedy w.r.t. current v (makes policy consistent with value).
- These two goals are *competing* (each step invalidates the other) but converge to a stable solution where $\pi = \text{greedy}(v_\pi) = \pi^*$.
- Every RL method — MC, TD, actor-critic — is a specific instance of GPI.

12 · Exam Key-Takeaways

Must Know — Sessions 4 & 5

1. **3 signals:** state S_t (env→agent), action A_t (agent→env), reward R_{t+1} (env→agent).
2. **Discount γ :** $\gamma = 0$ myopic; $\gamma = 1$ only for episodic; $0 < \gamma < 1$ in practice.
3. **Episodic vs continuing:** episodic has terminal state T ; continuing needs $\gamma < 1$ for convergence.
4. **G_t recursion:** $G_t = R_{t+1} + \gamma G_{t+1}$ — memorise and be able to derive.
5. **Bellman equation:** $v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) [r + \gamma v_\pi(s')]$.
6. **Policy Eval:** use Bellman expectation iteratively for given π .
7. **Value Iter:** use Bellman *optimality* — $\max_a$ replaces $\sum_a \pi(a|s)$.
8. **Policy Iter vs Value Iter:** Policy Iter waits for full eval; Value Iter doesn't.
9. **GPI:** any interleaving of evaluation + improvement converges to π^* .
10. **Async DP:** update states in any order; still needs full model; useful for large $|S|$.

DRL EC3 Exam Handout — Sessions 4 & 5 (MDP & Dynamic Programming). Ref: Sutton & Barto Ch. 3–4.



S1-2026-27 Work Integrated Learning Programmes S1-2026-27